

DECREASE AND CONQUER

Decrease by $(n/2)$



SCHOOL OF
INFORMATION TECHNOLOGY



BINARY SEARCH

- **Problem Statement**
 - Given the sorted list:
 - $A = [3, 7, 12, 18, 25, 31, 42, 56]$
 - Search for the key:
 - Key = 25



BINARY SEARCH (CONT.)

- **Binary Search reduces the problem size by half at every step.**
- **At each iteration:**
 - Compare the key with the middle element
 - Eliminate half of the search space
 - Solve the same problem on the remaining half
- **Only one smaller subproblem remains.**
- **This is the defining feature of decrease-by-a-constant-factor ($n / 2$).**





BINARY SEARCH (CONT.)

- **Initial Array**
 - [3, 7, 12, 18, 25, 31, 42, 56]
 - Problem size: $n = 8$



BINARY SEARCH (CONT.)

- **Step 1 – Find the Middle Element**

- Middle index:
 - $\text{mid} = (0 + 7) / 2 = 3$
 - $A[3] = 18$
- Comparison:
 - $25 > 18$
- Eliminate the left half
- Remaining subproblem:
 - $[25, 31, 42, 56]$

- **Problem size reduced from 8 $\rightarrow$ 4**





BINARY SEARCH (CONT.)

- **Step 2 – Find New Middle**
 - $\text{mid} = \text{index of } 31$
 - $A[\text{mid}] = 31$
 - Comparison:
 - $25 < 31$
 - Eliminate the right half
 - Remaining subproblem:
 - $[25]$
- **Problem size reduced from 4 $\rightarrow$ 1**





BINARY SEARCH (CONT.)

- **Step 3 – Final Comparison**
 - $A[\text{mid}] = 25$
 - MATCH FOUND
- **Final Answer**
 - Key 25 found





WHY THIS IS DECREASE-AND-CONQUER ($N / 2$)

- **At each step:**
 - One comparison is made
 - Half of the remaining elements are discarded
 - The same problem is solved on the remaining half
- **Mathematically:**
 - $T(n) = T(n / 2) + O(1)$
- **This is a decrease-and-conquer algorithm with reduction factor $n / 2$**



BINARY SEARCH (CONT.)

Aspect	Complexity
Time	$O(\log n)$
Space (Iterative)	$O(1)$
Space (Recursive)	$O(\log n)$





FAST EXPONENTIATION (REPEATED SQUARING)

- **Problem**
 - Compute:
 - 2^{10}
- **Key Idea ($n / 2$)** Instead of multiplying 2 ten times:
 - Reduce the exponent by half
 - Square intermediate results





FAST EXPONENTIATION (REPEATED SQUARING) (CONT.)

- **Step-by-Step Execution**

- $2^{10} = (2^5)^2$
- $2^5 = (2^2)^2 \times 2$
- $2^2 = (2^1)^2$





FAST EXPONENTIATION (REPEATED SQUARING) (CONT.)

- $2^1 = 2$
- $2^2 = 4$
- $2^5 = 4 \times 4 \times 2 = 32$
- $2^{10} = 32 \times 32 = 1024$



WHY THIS IS $N / 2$?

- Exponent reduces from $n \rightarrow n/2$ at each step
- Only one subproblem is solved
- Same problem form repeated





FAST EXPONENTIATION (REPEATED SQUARING) (CONT.)

- **Time Complexity**
 - $O(\log n)$





TOURNAMENT METHOD FOR MAXIMUM (HALVING CANDIDATES)

- **Problem**
 - Find the maximum in:
 - $A = [16, 10, 24, 7, 31, 18, 3, 28]$





TOURNAMENT METHOD FOR MAXIMUM (HALVING CANDIDATES)(CONT.)

- **Step-by-Step Execution**

- Round 1 – Pair comparisons ($n \rightarrow n/2$)
- (16 vs 10) $\rightarrow$ 16
- (24 vs 7) $\rightarrow$ 24
- (31 vs 18) $\rightarrow$ 31
- (3 vs 28) $\rightarrow$ 28

- **Remaining:**

- [16, 24, 31, 28]





TOURNAMENT METHOD FOR MAXIMUM (HALVING CANDIDATES)(CONT.)

- **Step-by-Step Execution**

- Round 2 – Pair comparisons ($n \rightarrow n/2$)
- (16 vs 24) $\rightarrow$ 24
- (31 vs 28) $\rightarrow$ 31

- **Remaining:**

- [24, 31]





TOURNAMENT METHOD FOR MAXIMUM (HALVING CANDIDATES)(CONT.)

- **Step-by-Step Execution**

- Round 3 – Pair comparisons ($n \rightarrow n/2$)
- (24 vs 31) $\rightarrow$ 31

- **Final Answer**

- [31]



WHY THIS IS $N / 2$

- Number of candidates halves each round
- Same comparison problem continues
- One subproblem per round





TOURNAMENT METHOD FOR MAXIMUM (HALVING CANDIDATES)(CONT.)

- **Time Complexity**
 - $O(\log n)$ rounds
 - $O(n)$ total comparisons



HALVING INTERVAL SEARCH (ROOT FINDING – CONCEPTUAL)

- **Problem**

- Find the root of $f(x)$ in the interval:
- $[0, 16]$

- **Method**

- Evaluate function at midpoint
- Keep only one half where sign changes
- Repeat

- **Execution**

- $[0, 16] \rightarrow$ midpoint 8 $\rightarrow$ choose $[0, 8]$
- $[0, 8] \rightarrow$ midpoint 4 $\rightarrow$ choose $[4, 8]$
- $[4, 8] \rightarrow$ midpoint 6 $\rightarrow \dots$





WHY THIS IS $N / 2$?

- Interval size halves each iteration
- One smaller subproblem remains
- Same structure repeated





EFFICIENT POWER-OF-TWO CHECK

- **Problem**
 - Determine if 64 is a power of two.
- **Execution**
 - $64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$
- **Repeatedly divide by 2 until base case reached.**





EFFICIENT POWER-OF-TWO CHECK

- **Why This Is $n / 2$?**
 - Value reduces by half each step
 - Same operation repeated
 - One subproblem at a time



END



SCHOOL OF
INFORMATION TECHNOLOGY